

DSA Lecture 4: Advance Sorting Algorithms (Merge & Quick)

1. Mergesort: The Stable Sort

Mergesort is a **stable** sorting algorithm that always achieves $O(n \log n)$ time complexity. It works by recursively dividing the array into halves, sorting them, and then **merging** the sorted halves.

Idea:

1. **Divide** the array into two halves until each part contains only one element.
2. **Conquer** by merging two sorted halves into a single sorted array.

Steps:

1. Split the array into two halves recursively.
2. Keep splitting until we have arrays of size 1.
3. Merge the sorted halves back together in the correct order.

Example:

Sorting [5, 3, 8, 4, 2] using Merge Sort:

Split:

```
[5, 3, 8, 4, 2]
=> [5, 3] & [8, 4, 2]
=> [5] & [3] | [8] & [4, 2]
=> [5] & [3] | [8] & [4] & [2]
```

Merge Back:

```
=> [3, 5] & [2, 4, 8]
=> [2, 3, 4, 5, 8] (final sorted array)
```

✓ **Time Complexity:** $O(n \log n)$

✓ **Stable** (Maintains order of duplicate elements)

1.1. Mergesort Implementation

C++

```

#include <iostream>
#include <algorithm> // For std::min and std::copy
using namespace std;

// Function to merge two sorted subarrays: arr[left...mid] and
arr[mid+1...right]
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int *leftArr = new int[n1];
    int *rightArr = new int[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    int i = 0; // Initial index of first subarray
    int j = 0; // Initial index of second subarray
    int k = left; // Initial index of merged array

    // Merge the temporary arrays back into arr[left...right]
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of leftArr[], if any
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
}

```

```

    // Copy the remaining elements of rightArr[], if any
    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }

    // Deallocate memory for temporary arrays
    delete[] leftArr;
    delete[] rightArr;
}

// Main recursive function that sorts arr[left...right]
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2; // Avoids overflow for
very large l and r

        // Divide (Recursive calls)
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Combine (Merge the sorted halves)
        merge(arr, left, mid, right);
    }
}

```

2. Quicksort: The In-Place Sort

Quicksort is generally the fastest practical sorting algorithm. It is an **in-place** sort, meaning it requires minimal extra space beyond the input array and the recursion stack.

Idea:

1. **Pick a pivot** (any element, usually first, last, or middle).
2. **Partition** the array so that elements smaller than the pivot go to the left and larger ones go to the right.
3. **Recursively** sort the left and right parts.

Steps:

1. Pick a pivot.
2. Move smaller elements to the left and larger ones to the right.
3. Recursively apply the same steps on both halves.

Example:

Sorting **[5, 3, 8, 4, 2]** using Quick Sort (choosing pivot = 5):

Partition:

```
[5, 3, 8, 4, 2]
Pivot = 5
Left: [3, 4, 2] | Pivot: [5] | Right: [8]
```

Recursively sort left and right parts:

```
Left: [3, 4, 2] → Pivot = 3 → [2] [3] [4]
Right: [8] (Already sorted)
```

Final sorted array: **[2, 3, 4, 5, 8]**

 **Time Complexity:** $O(n \log n)$ (best/average) but $O(n^2)$ (worst case if pivot is bad)

 **Not Stable** (Might change order of duplicate elements)

2.1. Quicksort C++ Array Implementation

Quicksort's efficiency comes from the **partition** function, which rearranges elements around a pivot. Since it modifies the array in place, no extra memory allocation is needed.

C++

```
int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // first element as pivot
    int i = low + 1;
    int j = high;

    while (true) {
        while (i <= high && arr[i] <= pivot) i++;
        while (j >= low && arr[j] > pivot) j--;

        if (i < j)
            swap(arr[i], arr[j]);
        else
            break;
    }

    swap(arr[low], arr[j]); // place pivot in correct position
    return j; // return pivot index
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

3. Full Program Example and Execution

This complete program demonstrates how to call both sorting functions on standard integer arrays.

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

// --- Utility Functions ---

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
```

```

}

// --- Mergesort Implementation (Re-included for compilation) ---

void merge(int arr[], int left, int mid, int right) {
    // ... (Implementation from section 1.1) ...
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *leftArr = new int[n1];
    int *rightArr = new int[n2];

    for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++) rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) arr[k] = leftArr[i++];
        else arr[k] = rightArr[j++];
        k++;
    }

    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];

    delete[] leftArr;
    delete[] rightArr;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// --- Quicksort Implementation (Re-included for compilation) ---

int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // first element as pivot
    int i = low + 1;

```

```

int j = high;

while (true) {
    while (i <= high && arr[i] <= pivot) i++;
    while (j >= low && arr[j] > pivot) j--;

    if (i < j)
        swap(arr[i], arr[j]);
    else
        break;
}

swap(arr[low], arr[j]); // place pivot in correct position
return j; // return pivot index
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

// --- Main Execution ---

int main() {
    int arr1[] = {12, 11, 13, 5, 6, 7};
    int n1 = sizeof(arr1) / sizeof(arr1[0]);
    cout << "Original Array (Mergesort): ";
    printArray(arr1, n1);

    mergeSort(arr1, 0, n1 - 1);
    cout << "Sorted Array (Mergesort): ";
    printArray(arr1, n1);

    cout << string(40, '-') << endl;

    int arr2[] = {10, 80, 30, 90, 40, 50, 70};
    int n2 = sizeof(arr2) / sizeof(arr2[0]);
    cout << "Original Array (Quicksort): ";
    printArray(arr2, n2);

    quickSort(arr2, 0, n2 - 1);
    cout << "Sorted Array (Quicksort): ";

```

```
    printArray(arr2, n2);  
  
    return 0;  
}
```